NETWORK PROGRAMMING · UTM · SUPORT DE CURS EXPLICAT PAS CU PAS (CURSURILE 1, 2 ȘI 3 — ERNEST BÎTCA)

Programare Concurentă, Sincronizare și Async/Await în C#

Ghid complet de la zero: de ce procesoarele au mai multe nuclee, cum funcționează firele de execuție, cum evităm bug-urile de concurență și cum scriem servere rapide.

CURS 1 1. De ce trebuie să învățăm asta? Sfârșitul „Prânzului Gratuit” (The Free Lunch)

Dacă nu înțelegi de ce complicăm programele cu *multithreading*, răspunsul se află în fizica procesoarelor. Timp de aproape 30 de ani, între **1975 și 2004**, programatorii au beneficiat de așa-numitul **„prânz gratuit” (The Free Lunch)**: puteai face un program lent să devină rapid fără să modifici nicio linie de cod. Frecvența ceasului procesorului (clock frequency) a crescut de la câțiva megahertzi (MHz) până la aproape **4 gigahertzi (GHz)**, astfel încât același program cu un singur fir de execuție (*single-threaded*) rula automat mai repede pe fiecare generație nouă de calculatoare.

- **Problema fizică din 2004 (Căldura):** O frecvență mai mare cere mai mult curent și generează mult mai multă căldură (*Faster clock → more power → more heat*). În jurul anului 2004, căldura a devenit imposibil de disipat; Intel a anulat procesorul Pentium 4 de 4 GHz, iar frecvențele au înghețat în jurul valorii de **4–5 GHz** până în prezent.
- **Hyper-Threading (Un muncitor cu două tăvițe de documente):** Imaginează-ți un nucleu de procesor (CPU core) ca pe un muncitor la un birou. În mod normal, el rezolvă o singură sarcină pe rând, dar deseori stă degeaba câteva nanosecunde așteptând să vină datele din memoria RAM. *Hyper-threading* îi oferă aceluiași muncitor **două tăvițe de intrare (inboxes)**: când sarcina A se blochează așteptând date din memorie, muncitorul comută instantaneu pe sarcina B. Aduce un câștig practic de doar **15–30%** (și aproape zero dacă un singur program ține deja muncitorul ocupat 100%).
- **Multi-Core (Mai mulți muncitori reali, fiecare cu biroul său):** Un procesor cu 4 nuclee (*4-core processor*) are literalmente **4 unități de calcul complete și independente**. Ele nu comută rapid între ele, ci lucrează cu adevărat simultan, putând oferi de aproape 4× mai multă viteză — dar **doar dacă programatorul scrie cod concurent** care să împartă munca!

CURS 1 2. Vocabularul de Bază: Cele 4 Concepte Explicate prin Analogii Vizuale

- **Concurența (Concurrency — „A te ocupa de mai multe lucruri în același interval de timp”):** Ai nevoie de concurență ori de câte ori o aplicație trebuie să răspundă la un click în timp ce salvează în baza de date. *Analogia 1 (Automatul de suc):* Două cozi de oameni în fața unui **singur automat de suc**; ambele cozi avansează alternativ, dar doar o singură persoană primește suc la o secundă dată. *Analogia 2 (Bucătarul):* **Un singur bucătar** are pastele la fiert pe aragaz, pâinea la copt în cuptor și taie salata pe tocător. Toate trei progresează în același interval de timp pentru că bucătarul nu stă niciodată pe loc.
- **Paralelismul / Procesarea Paralelă (Parallelism — „A face mai multe lucruri exact în aceeași clipă”):** *Analogia 1:* Două cozi de oameni în fața a **două automate de suc** — doi oameni sunt serviți în aceeași clipă. *Analogia 2 (Farfuriile):* Un teanc mare de farfurii murdare este împărțit în 4 sferturi egale la **4 oameni** care le spală simultan, terminând într-un sfert din timp.
- **Multithreading (Mai multe fire de execuție):** O formă de concurență în care mai multe fire lucrează în același proces. *Analogia bucătăriei:* **Trei bucătari** lucrează simultan în aceeași bucătărie și împart **un singur frigider comun**. Pericolul apare când doi bucătari întind mâna după același ou în aceeași clipă!
- **Programarea Asincronă (Asynchronous Programming):** Concurență care folosește promisiuni (*futures / Task*) sau *callbacks* pentru a evita blocarea inutilă a firelor. *Analogia cuptorului cu microunde:* (1) Apeși Start, (2) Pleci și faci altceva în bucătărie, (3) Te întorci când auzi bip-ul. A sta încremenit și a te holba la microunde timp de 3 minute este exact ceea ce face un **thread blocat**: un om întreg care nu face absolut nimic!



CURS 1 3. Program vs. Proces vs. Thread: Ce este Comun și ce este Privat?

- **01 · Programul (Program):** Este o simplă secvență de instrucțiuni scrisă într-un limbaj de programare și salvată ca un **fișier pe disc** (ex. `server.exe`). Nu consumă memorie RAM și nimic nu rulează încă.
- **02 · Procesul (Process — Deținătorul resurselor):** Când dai dublu-click pe program, sistemul de operare (OS) creează un **proces** — o instanță în execuție care primește propriul său spațiu de memorie virtuală izolat. Procesul deține resursele: **Codul (Code)**, **Datele globale și memoria dinamică (Heap & static fields)**, **fișierele și conexiunile de rețea deschise (Files & Socket handles)** și are cel puțin un thread.
- **03 · Firul de execuție (Thread — Unitatea de planificare / scheduling):** Un thread este un flux individual de execuție în interiorul unui proces. Într-un proces *multithreaded*, toate firele **împart la comun aceeași memorie Heap, aceleași variabile statice și aceleași fișiere deschise**. În schimb, fiecare thread are **propriul său set de Registre (Registers)**, propriul **Program Counter** (ca să știe la ce instrucțiune a ajuns) și **propria sa Stivă (Stack)**. Prin urmare, **variabilele locale aparțin exclusiv unui singur thread** și nu pot fi văzute de celelalte fire!
- **CPU Threads vs. Software Threads:** Un *CPU thread* (thread hardware) este un slot fizic sau logic oferit de procesor pentru a executa instrucțiuni (ex. 4 nuclee cu Hyper-Threading = 8 CPU threads). Un *Software thread* este un obiect creat din codul nostru C# și gestionat de sistemul de operare; poți crea 1.000 de thread-uri software chiar dacă procesorul are doar 4 sau 8 thread-uri hardware.

CURS 1 4. Primul Cod C# cu Thread-uri și Costul Ascuns al unui Thread

```
// PASUL 1: Descriem munca pe care vrem să o execute noul fir (îi dăm metoda Work)
Thread worker = new Thread(Work);

// PASUL 2: Pornim firul! Din această clipă, DOUĂ fire rulează simultan (Main și worker)
worker.Start();

// PASUL 3: Obligăm firul principal (Main) să aștepte aici până când worker termină
worker.Join();

void Work() // Această metodă se execută pe un thread diferit față de Main!
{
    int id = Thread.CurrentThread.ManagedThreadId;
    Console.WriteLine($"[worker] rulează pe thread-ul cu ID-ul {id}");
}
```

De reținut din laborator (Module 1): Ordinea în care pornesc și afișează thread-urile **nu este garantată** de sistemul de operare! În plus, **înainte să creezi un thread, trebuie să știi cât te costă:** un singur thread consumă din start **1 MB de memorie RAM rezervată pentru stivă (reserved stack)** + un obiect de kernel + un *Thread Environment Block* + un loc permanent în cozile planificatorului OS + zeci de microsecunde doar pentru creare. Dacă ai un server cu **10.000 de clienți** și creezi câte un thread pentru fiecare, consumi **10.000 × 1 MB = 10 GB de spațiu de adrese** înainte ca programul tău să fi stocat măcar un singur byte de date utile! Acesta este motivul existenței lui `async / await` din Cursul 3.

CURS 1 5. CPU-Bound vs. I/O-Bound: Două Tipuri de „Lent” cu Soluții Opuse

Când un program este lent, trebuie să aplici **Testul Suprem**: *„Dacă aduc mai mulți oameni (nuclee/thread-uri) să lucreze, se termină mai repede?”* Dacă **DA** → **este CPU-Bound** (cureți 100 de cartofi; tu muncești continuu și tu ești blocajul). Dacă **NU** → **este I/O-Bound** (ai comandat o pizza care vine în 30 de minute; stai degeaba și aștepți o sursă externă — dacă chemi încă 5 prieteni în cameră, pizza tot în 30 de minute sosește!).

Criteriu de Comparație	CPU-BOUND (Limitat de Procesor)	I/O-BOUND (Limitat de Intrare / Ieșire)
Ce îl limitează?	Viteza cu care procesorul (CPU) poate face calcule.	**Ceva din afara procesorului (rețea, disc, bază de date).**
Ce face nucleul CPU?	Este ocupat aproape **100% din timp** cu calcule intense.	**Stă inactiv (*idle*) aproape tot timpul, așteptând date.**
Analogie din viața reală	Curățarea a 100 de cartofi (muncești fără oprire)	**Așteptarea unei pizza comandate (30 minute de așteptare).**

CURS 2 6. The Thread Pool: Trimiți de Lucru, Nu Creezi Thread-uri Noi

```
// În Cursul 1: TU deții thread-ul, TU plătești 1 MB și costul de creare pentru el
Thread t = new Thread(Work);
t.Start();

// Acum (Cursul 2): TU deții doar munca, iar runtime-ul .NET deține și reciclează thread-urile
ThreadPool.QueueUserWorkItem(_ => Work());

// Forma modernă pentru exact același lucru (împrumută un thread din bazin, NU creează unul nou!)
Task.Run(() => Work());
```

Cum funcționează Thread Pool-ul și Regula sa de Aur: Deoarece thread-urile sunt scumpe, .NET creează o singură dată un grup de muncitori (*Worker Threads*) pe care îi ține în viață și îi refolosește. Tu pui sarcini într-o coadă (*Work Queue*), iar muncitorii liberi le preiau una câte una. **REGULA SUPREMĂ: Nu bloca niciodată un thread din Thread Pool (Never block a pool thread)!** Bazinul are un număr limitat de fire și este proiectat pentru sarcini scurte și rapide. Dacă blochezi un muncitor așteptând ceva lent, el nu mai poate servi pe nimeni altcineva; dacă îi blochezi pe toți, întreaga aplicație îngheață, chiar dacă codul este „corect” logic!

CURS 2 7. Problema Corectitudinii: De ce `counter++` NU Este o Singură Operație!

Cursul 1 a fost despre viteză; Cursul 2 este despre **corectitudine** — pentru că un program poate fi foarte rapid și complet greșit! În Cursul 1 am văzut că toate thread-urile dintr-un proces împart memoria **Heap** și că sistemul de operare poate opri un thread între oricare două instrucțiuni. Ce se întâmplă dacă pornim **8 thread-uri** care incrementează aceeași variabilă `int counter = 0;` de câte `1_000_000` de ori fiecare (`for (int k = 0; k < 1_000_000; k++) counter++;`)?

Ne-am aștepta ca la final `Console.WriteLine(counter)` să afișeze exact **8.000.000**. În realitate, afișează un număr mult mai mic! De ce? Pentru că în limbajul mașină al procesorului, `counter++` **NU este o singură operație, ci TREI operații separate**: **(1) READ** (citește valoarea din RAM în registru), **(2) ADD** (adună 1 la acea valoare), **(3) WRITE** (scrie rezultatul înapoi în RAM). Dacă sistemul de operare întrerupe Thread A după ce a citit `41`, iar Thread B intră și citește tot `41`, ambele calculează `42` și ambele scriu `42`. Au avut loc două incrementări, dar contorul a crescut doar cu 1 (**Lost Update / Actualizare pierdută**)!

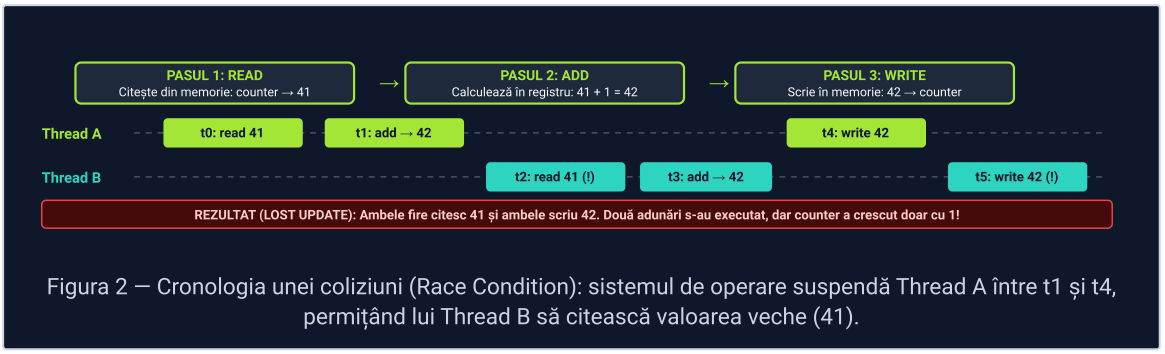


Figura 2 — Cronologia unei coliziuni (Race Condition): sistemul de operare suspendă Thread A între t1 și t4, permițând lui Thread B să citească valoarea veche (41).

CURS 2 8. Definiția unui Race Condition și De Ce Testele NU îl Prind

- **Ce este un Race Condition (Condiție de cursă):** Apare atunci când corectitudinea unui program depinde de ordinea și sincronizarea în timp a thread-urilor — lucruri pe care nimeni nu le controlează. Pentru a exista, un Race Condition are nevoie obligatoriu de **două ingrediente simultan**: **(1) Stare mutabilă partajată (shared mutable state)** și **(2) Mai mult de un thread care o accesează**. Dacă elimini oricare dintre aceste două ingrediente, condiția de cursă devine imposibilă!
- **De ce testele tale automate NU îl vor depista:** Rezultatul unui program cu race condition nu este aleatoriu; este *mai periculos decât aleatoriu: de obicei dă rezultatul corect!* În experimentul din curs, rulând exact același program defect cu volume diferite de muncă, obținem: la **1.000 iterații** → așteptat `1.000`, obținut `1.000` (**CORECT**); la **100.000 iterații** → așteptat `100.000`, obținut `100.000` (**CORECT**); la **1.000.000 iterații** → așteptat `1.000.000`

CURS 2 9. Scara Soluțiilor („The Ladder of Fixes”): Începe Mereu de Jos!

Când ai un *race condition*, cea mai bună reparație nu este un lacăt (`lock`) mai complicat, ci **să nu împărți datele**. Urcă pe această scară doar când nu poți rămâne pe o treaptă mai ieftină:

1. **TREAPTA 1 — DON'T SHARE (Nu partaja datele · Cost: GRATUIT):** Dă fiecărui thread propria sa bucată de memorie unde nimeni altcineva nu scrie. De exemplu, creezi un vector `int[] counts = new int[threadCount];` în care thread-ul `i` scrie exclusiv în căsuța sa `counts[index] = Count(...);` . Abia la final, după ce toate firele au terminat (după `Join()`), aduni sumele parțiale cu `int total = counts.Sum();` .
2. **TREAPTA 2 — MAKE IT IMMUTABLE (Fă obiectul imutabil · Cost: GRATUIT):** Dacă nimic nu se poate schimba după crearea obiectului, nimic nu poate fi corupt! Într-o clasă `class Reading { public string Sensor { get; } public double Value { get; } ... }` fără `set` , valorile sunt setate o singură dată în constructor și înghețate pe viață. Oricâte thread-uri o pot citi simultan fără niciun risc.
3. **TREAPTA 3 — `Interlocked` (Operații atomice pe o singură variabilă · Cost: IEFTIN, fără blocare):** Când trebuie neapărat să modifici o singură variabilă primitivă partajată, folosești instrucțiuni hardware care execută *read-modify-write* ca **un singur pas indivizibil**: `Interlocked.Increment(ref counter);` , `Interlocked.Add(ref total, 42);` sau `Interlocked.Exchange(ref flag, 1);` .
4. **TREAPTA 4 — `lock` (Protejarea unei secțiuni întregi de cod · Cost: SCUMP):** Când două sau mai multe operații trebuie să se întâmple împreună (ex: `balance -= amount; history.Add(transaction);` — lucru pe care `Interlocked` nu îl poate face), le închizi într-un bloc protejat de un obiect santinelă: `private readonly object _gate = new(); lock (_gate) { ... }` . Doar un singur thread poate intra în bloc la un moment dat; toate celelalte așteaptă la coadă.
5. **Cele 4 Reguli Sfinte pentru `lock` : (1)** Blochează mereu pe un obiect `private readonly object` (niciodată `lock(this)` și niciodată pe un `string` , fiindcă alt cod din afară îl poate bloca și el!); **(2)** Ține lacătul ocupat cât mai puțin timp posibil (înăuntrul lui, programul tău redevine single-threaded!); **(3)** Nu face niciodată operații I/O (rețea, fișiere) în interiorul unui `lock` ; **(4)** Absolut fiecare citire sau scriere a acelei variabile trebuie să folosească exact același obiect `lock` (o singură scriere neprotejată face toate celelalte lacăte inutile!).

CURS 2 10. Când Blocarea Merge Prost (Liveness) și Primitivele Care Nu Sunt Lacăte

Folosirea greșită a lacătelor produce trei blocaje clasice de *liveness*: **(1) Deadlock (Blocaj mortal):** Thread A ia `lock(first)` , așteaptă 50ms și cere `lock(second)` , în timp ce Thread B ia `lock(second)` și cere `lock(first)` . Fiecare așteaptă după celălalt la infinit (CPU stă la 0%, progres zero). **(2) Livelock:** Două fire „prea politicoase”: Thread 1 ia `lockA` și încearcă `lockB.TryEnter()` ; văzând că e ocupat de Thread 2, eliberează `lockA` , așteaptă 50ms și reîncearcă — dar Thread 2 face exact același lucru în oglindă, în același moment (la 0ms, 50ms, 100ms, 150ms...). CPU-ul muncește intens, dar nimeni nu avansează! **(3) Starvation (Înfometare):** Un thread „lacom” (*greedy*) face o operație lentă (`Thread.Sleep(1)`) **înăuntrul** blocului `lock(gate)` ; deoarece `lock` nu este o coadă echitabilă, când îl eliberează, tot firul care deja rula pe CPU îl recaptură imediat (1.751 ture pentru cel lacom vs. doar 23 ture pentru celălalt!). Dacă muți `Thread.Sleep(1)` în afara lui `lock` , ambele primesc ~1.753 ture.

În fine, **nu orice primitivă este un lacăt** — există trei instrumente care răspund la trei întrebări complet diferite: **1. WHO? (Cine are voie să intre?)** → **`lock`** (1 singur thread pe rând, protejează starea partajată; `Mutex` face același lucru între procese diferite, dar este de ~50× mai lent). **2. HOW MANY? (Câte fire au voie simultan?)** → **`SemaphoreSlim`** : limitează concurența la cel mult *N* fire simultan (ex: `var gate = new SemaphoreSlim(3); gate.Wait(); try { DoWork(); } finally { gate.Release(); }` — ideal pentru a limita descărcările sau conexiunile la baza de date). **3. WHEN? (Când trecem mai departe?)** → **`Barrier`** : sincronizează lucrul pe faze (ex: `Barrier barrier = new Barrier(4, ...);` unde fiecare dintre cei 4 muncitori apelează `barrier.SignalAndWait();` la finalul Fazei 1; cine termină mai devreme așteaptă, și nimeni nu începe Faza 2 până nu au ajuns toți 4!).

Problemă / Primitivă	Ce fac Thread-urile?	Stare CPU	Cine face progres?	Cauza Tipică & Soluția Corectă în C#
Deadlock	Așteaptă unul după celălalt în cerc.	Inactiv (Idle)	**Nimeni (blocat total)**	Lacăte luate în ordine opusă (A→B vs B→A). **Fix:** Impune aceeași ordine a lacătelor peste tot

CURS 3 11. Ce Face de Fapt `async` / `await` cu Thread-ul? Așteptare fără Blocare

În Cursul 2 am învățat că nu avem voie să blocăm firele din Thread Pool, dar lacătele, semafoarele și apelurile de rețea clasice tocmai asta fac: țin firul blocat în timp ce așteaptă. Cum poate un server să aștepte după rețea fără să țină niciun thread ocupat? Răspunsul este `async / await`:

```
// VARIANTA 1 (BLOCANTĂ): Thread-ul 1 este ținut ostatic 500 ms și nu poate face nimic altceva
Console.WriteLine($"before Sleep : thread {Environment.CurrentManagedThreadId}"); // -> thread 1
Thread.Sleep(500);
Console.WriteLine($"after Sleep  : thread {Environment.CurrentManagedThreadId}"); // -> thread 1

// VARIANTA 2 (ASINCRONĂ): Niciun thread nu așteaptă! Firul 1 este eliberat în bazin!
Console.WriteLine($"before await : thread {Environment.CurrentManagedThreadId}"); // -> thread 1
await Task.Delay(500);
Console.WriteLine($"after await  : thread {Environment.CurrentManagedThreadId}"); // -> thread 5 (!)
```

Ce se întâmplă sub capotă în 3 pași (Under the Hood): (1) Rulează normal: Până la primul cuvânt `await`, o metodă `async` rulează exact ca orice altă metodă obișnuită, pe thread-ul apelantului (ex. `thread 1`). **(2) Pune pauză și eliberează firul:** Când ajunge la `await`, dacă operația (descărcarea, timer-ul, citirea) nu este gata, metoda își memorează locul unde s-a oprit și se întoarce imediat; **thread-ul 1 devine liber** și se întoarce în Thread Pool ca să servească alți clienți, în timp ce placa de rețea și sistemul de operare așteaptă datele (fără niciun thread!). **(3) Continuă mai târziu:** Când datele sosesc, restul metodei își reia execuția exact de unde rămăsese, pe oricare thread liber din bazin (ex. `thread 5`). **Important:** `async` NU pornește un thread nou; el permite unei metode să aștepte fără să țină ocupat un thread!

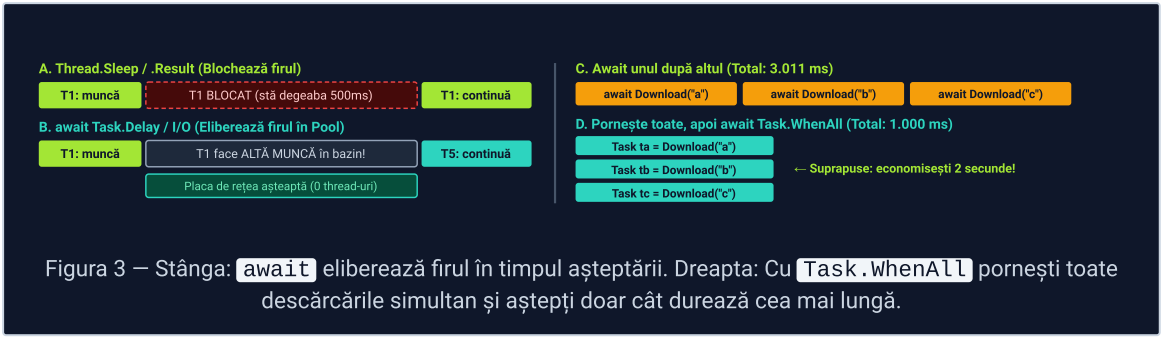


Figura 3 — Stânga: `await` eliberează firul în timpul așteptării. Dreapta: Cu `Task.WhenAll` pornești toate descărcările simultan și aștepți doar cât durează cea mai lungă.

CURS 3 12. `Task.WhenAll`, Scalabilitatea Serverelor și Cele 3 Reguli pentru `await`

- **Pornește întâi, așteaptă mai târziu (`Task.WhenAll`):** Dacă scrii `string a = await DownloadAsync("a"); string b = await DownloadAsync("b"); string c = await DownloadAsync("c");` (unde fiecare durează 1 secundă), a doua descărcare nu începe până nu se termină prima, deci aștepți în total **3.011 ms**. Dacă în schimb pornești întâi toate cele trei task-uri fără `await` (`Task<string> ta = DownloadAsync("a"); Task<string> tb = DownloadAsync("b"); Task<string> tc = DownloadAsync("c");`) și apoi le aștepți împreună cu `string[] all = await Task.WhenAll(ta, tb, tc);`, așteptările se suprapun și totul se termină în **1.000 ms**!
- **De ce le pasă serverelor (Experimentul cu 1.000 de clienți care așteaptă 1 secundă):** Când simulăm 1.000 de clienți cu `Thread.Sleep(1000)`, programul termină în `1.123 ms`, dar creează **1.009 thread-uri OS** în proces! Când simulăm aceiași 1.000 de clienți cu `await Task.Delay(1000)`, programul termină în `1.006 ms` folosind doar **11 thread-uri OS** (de ~100 de ori mai puține fire!). Așa reușește un singur server modern să țină zeci de mii de conexiuni deschise simultan.
- **I/O-Bound vs. CPU-Bound cu `await`:** `await` nu face munca mai rapidă, ci face **așteptarea ieftină**! Pentru operații **I/O-Bound** (rețea, disc, timere), apelezi direct metoda asincronă: `string text = await File.ReadAllTextAsync(path);` (niciun thread nu este ținut în timp ce datele călătoresc). Pentru operații **CPU-Bound** (parsare, sortare, calcul numere prime), care au nevoie de un nucleu CPU tot timpul, folosești `int primes`

CURS 3 13. Colecții Thread-Safe și Arhitectura Producer–Consumer

În Cursul 2 am văzut că o listă standard `List<T>` sau un `Dictionary<K,V>` pierde elemente sau aruncă excepții când mai multe task-uri adaugă date simultan fără `lock`. În loc să pui tu manual lacăte peste tot (Treapta 5 a scării), .NET oferă în `System.Collections.Concurrent` colecții construite special pentru acces concurent:

- **Cele 4 Colecții Thread-Safe esențiale: (1) `ConcurrentQueue<T>`:** Coadă *First-In, First-Out (FIFO)* — elementele ies exact în ordinea în care au intrat; **(2) `ConcurrentStack<T>`:** Stivă *Last-In, First-Out (LIFO)* — ultimul intrat este primul servit; **(3) `ConcurrentBag<T>`:** Coș fără nicio ordine garantată — extrem de rapid când aceleași task-uri adaugă și extrag elemente (4 task-uri pot umple un `ConcurrentBag` fără niciun `lock` și nu se pierde niciun element!); **(4) `ConcurrentDictionary<K, V>`:** Dicționar cheie → valoare în care **fiecare apel individual de metodă este atomic**.
- **Capcana majoră la `ConcurrentDictionary` („Un singur apel, nu două!"):** Deși fiecare apel individual este atomic, **două apeluri puse unul după altul NU sunt atomice împreună!** Cele 3 erori clasice pe care trebuie să le eviți:
  1. *Valoarea nouă depinde de cea veche:* `d[k] = d[k] + 1;` (**GREȘIT** — sunt două apeluri: o citire `d[k]` și o scriere; alt task poate modifica valoarea exact între ele! Folosește `d.AddOrUpdate(...)`).
  2. *Verifici, apoi citești:* `if (d.ContainsKey(k)) v = d[k];` (**GREȘIT** — alt thread poate șterge cheia exact după ce `ContainsKey` a dat `true`! Folosește un singur apel: `if (d.TryGetValue(k, out var v))`).
  3. *Verifici, apoi scrii:* `if (!d.ContainsKey(k)) d[k] = v;` (**GREȘIT** — două task-uri pot vedea simultan că lipsește cheia și amândouă vor suprascrie! Folosește `d.TryAdd(k, v)` sau `d.GetOrAdd(k, v)`).
- **Șablonul Producer–Consumer (Designul din spatele Lab 1c):** Unul sau mai multe fire **Producătoare (Producers)** generează pachete de lucru și le adaugă (`Add` / `Enqueue`) într-o coadă partajată, în timp ce firele **Consumatoare (Consumers)** extrag pachetele din coadă (`Take`) pe măsură ce devin libere (folosind de exemplu `BlockingCollection<T>` sau `Channel<T>`). Coada acționează ca un amortizor sigur între ele.

CURS 3 14. Cele 4 Moduri de a Aștepta un Eveniment și Cuvântul Cheie `volatile`

Să presupunem că un thread trebuie să aștepte ceva care se va întâmpla peste 2 secunde. Iată cele 4 moduri în care poate aștepta și costul fiecăruia:

Metodă de Așteptare	Cod C# Ilustrativ	Consum Procesor (CPU)	Comportament & Verdict Practic
1. Busy Wait (Așteptare activă)	`while (!ready) { }`	**100% dintr-un nucleu CPU!**	Verifică variabila de milioane de ori pe secundă. Ține un nucleu întreg ocupat 100% făcând nimic util.
2. Sleep in a Loop	`while (!ready) Thread.Sleep(50);`	Aproape 0% CPU	Nu mai arde procesorul, dar ține thread-ul blocat și reacționează cu o întârziere de până la **50 ms**.
3. Blocking Wait	`ready.Wait();`	0% CPU (firul doarme)	Reacționează imediat și nu consumă CPU, dar **ține un thread blocat** pe toată durata celor 2 secunde.
4. Async Wait (Ideal)	`await readyTask;`	**0% CPU + 0 fire blocate!**	**Soluția modernă: metoda pune pauză, iar thread-ul este eliberat imediat în bazin pentru a servi alți clienți.**

Ce face și ce NU face cuvântul cheie `volatile` (Capcană frecventă la examen!):

- **CE FACE `volatile`:** Obligă procesorul și compilatorul ca fiecare citire a acelui câmp să citească valoarea **direct din memorie**, fără să o păstreze ascunsă (*cached*) într-un registru local al nucleului. Este util doar atunci când un thread stă într-o buclă `while (!stopFlag)` și alt thread setează `stopFlag = true`.
- **CE NU FACE `volatile`: (1) NU face `counter++` atomic!** Citirea, adunarea și scrierea rămân tot 3 pași separați, deci un `volatile int counter` va avea în continuare *race condition* (folosește `Interlocked.Increment`!). **(2) NU face așteptarea ieftină!** Un *busy wait* (`while (!ready) {}`) pe un câmp `volatile` tot consumă 100% dintr-un nucleu de procesor!